

Fall 2026 Exploratory Data Analysis with R

Lecture 5: Plots, Dates, and Strings

Brian Robertson
Federal Reserve Bank of Atlanta
2026-09-25

01

Recap & Agenda

01 Recap & Agenda

02 Visualizing Data

03 Dates in R

04 Manipulating Strings

05 Tidyverse Review

06 Research Process & The Project

07 Recap

Quick Recap: Last Week

- Practiced **code hygiene** — organizing a script, commenting the *why*, and naming for clarity
- Reached for `case_when()` instead of a nested `if_else()` mess for multi-way conditions
- Picked up basic **debugging** — reading the error message and isolating a broken pipe
- Knit text, code, and results together in **R Markdown**, and formatted tables with `kable()`

Today: a new module. We turn from summarizing data to *seeing* it — plus two data types, dates and strings, that show up in almost every real dataset.

Goals for Today

- Build a **visual vocabulary**: match a data relationship (distribution, change over time, magnitude, relationship between variables) to the right `ggplot2` geometry
- Understand how R actually **stores dates**, and convert raw date strings into date objects with `lubridate`
- **Manipulate strings** — extract, replace, and split text, and get a first look at regular expressions
- Review the **tidyverse** toolkit from Lectures 2 and 4
- Connect today's tools to **Project Update #3**

02

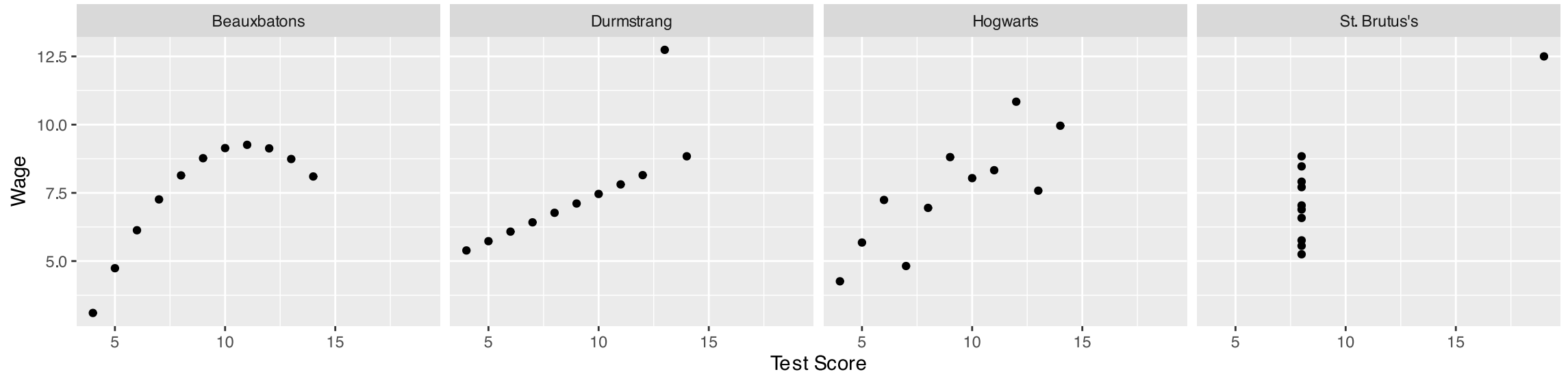
Visualizing Data

- 01 Recap & Agenda
- 02 Visualizing Data**
- 03 Dates in R
- 04 Manipulating Strings
- 05 Tidyverse Review
- 06 Research Process & The Project
- 07 Recap

Why Visualize?

- A picture provides different information than summary statistics — combine the two to build a case
- Visualizations let you understand your data quickly, and communicate results to *anyone*, not just readers of a regression table
- A good chart **clarifies your story** — it shows you what you need to make an argument, and shows your reader what you found

Summary Statistics Alone Aren't Enough

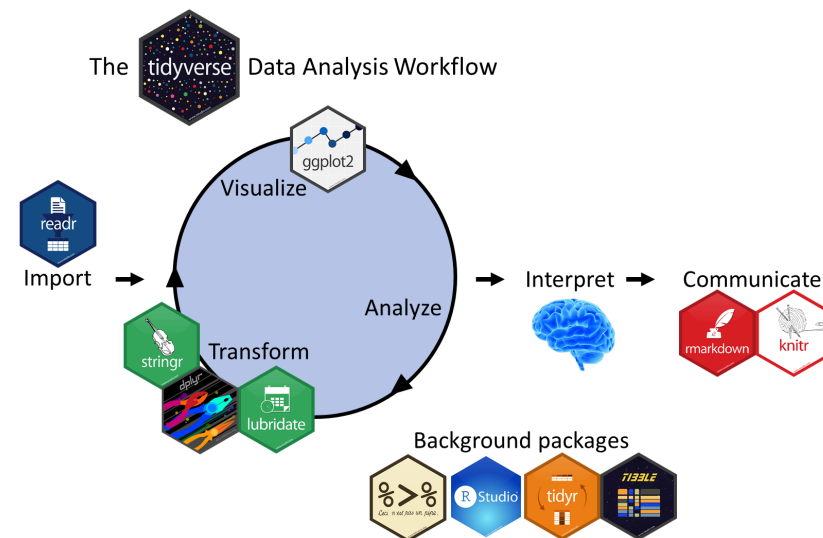


All four of these datasets share the same mean, variance, correlation, and regression line — [Anscombe's quartet](#). Summary statistics alone would tell you they're identical.

Building a Visual Vocabulary

Every visualization breaks down into three elements — what Hadley Wickham calls the “grammar of graphics”:

- **Data** — the underlying data points
- **Mapping** — how a variable is represented: which aesthetic (x/y axis, size, color) it’s assigned to
- **Geometry** — the visual form itself (point, line, bar, ...)



The Syntax of `ggplot()`

Three steps, every time:

1. Initialize a `ggplot` object with your **data**
2. Define the **aesthetic mappings** (`aes()`) between variables and axes
3. Add a **layer of geometry** (`geom_*()`)



Basic template: `ggplot(data = <DATA>, mapping = aes(<MAPPINGS>)) + <GEOM_FUNCTION>()`

There are thousands of ways to visualize data — the question we'll answer four times today is: **which relationship are you showing?**

- Distribution
- Change over time
- Magnitude
- Relationship between two variables

[ggplot_gallery](#) · [ggplot_cheat_sheet](#)

Distribution

Shows how often values occur — e.g. an income distribution, or an age distribution.

- **Histograms** are the standard way to show a distribution: bin the values, and make bar heights proportional to how many observations fall in each bin
- Recall **spread** and **location** from Lecture 3 — a histogram is where you see both at once

Building a Histogram, Step by Step

How is `education` distributed in `BankWages`?

1. Data

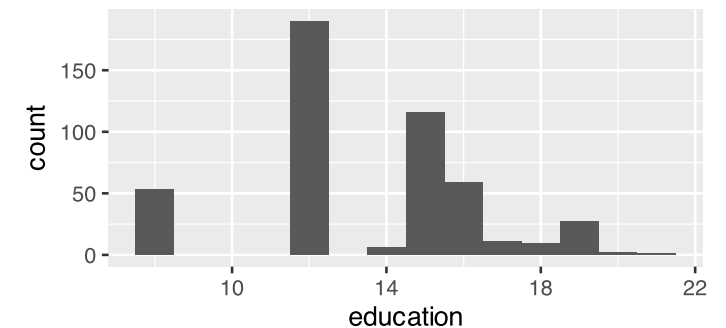
```
ggplot(BankWages)
```

2. Mapping

```
ggplot(BankWages,  
  aes(x = education))
```

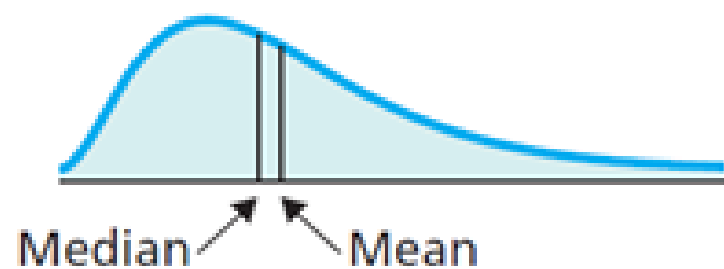
3. Geometry

```
ggplot(BankWages,  
  aes(x = education)) +  
  geom_histogram(binwidth =  
    1)
```

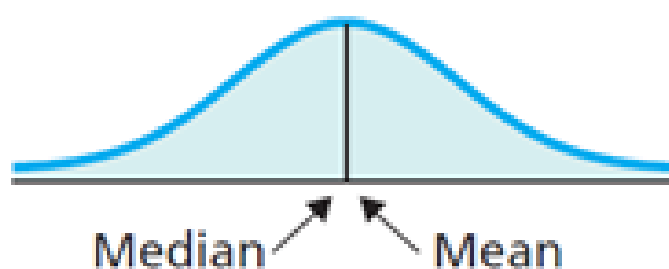


Reading the Shape

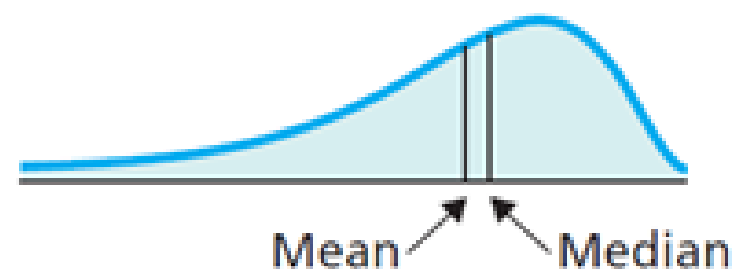
- Mean > median tells us this distribution is **positively skewed** — a “long right tail”
- Why so lumpy, with masses at 12, 15, and 16 years? Education is typically recorded by **credential** (high school diploma, bachelor’s degree), not a smooth count of years



Positively skewed



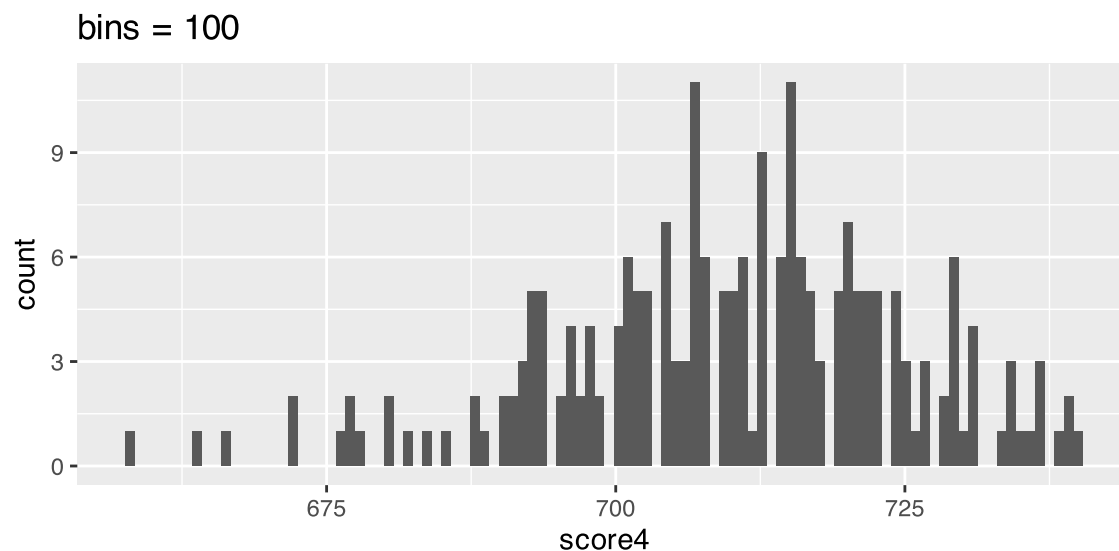
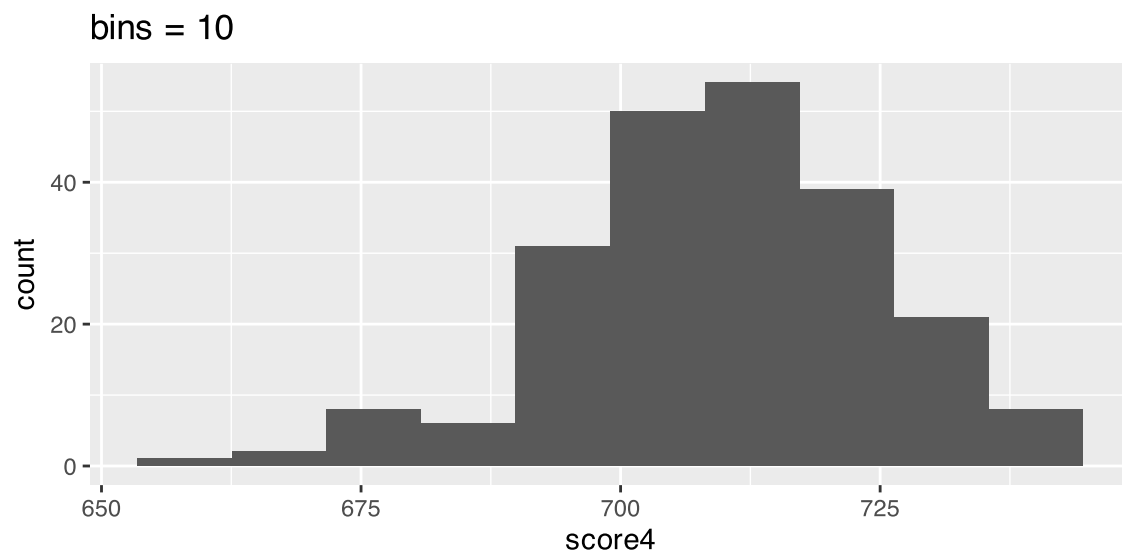
Normal Distribution



Negatively skewed

Bins and Binwidths

Histograms are parameterized by their **number of bins** (`ggplot`'s default is 30) or the **width of a bin** — set with `geom_histogram(bins = x)` or `geom_histogram(binwidth = x)`.



Too few bins and you blend away real structure; too many, and there's nothing left to summarize.
(`MASchools$score4`: fourth-grade test scores.)

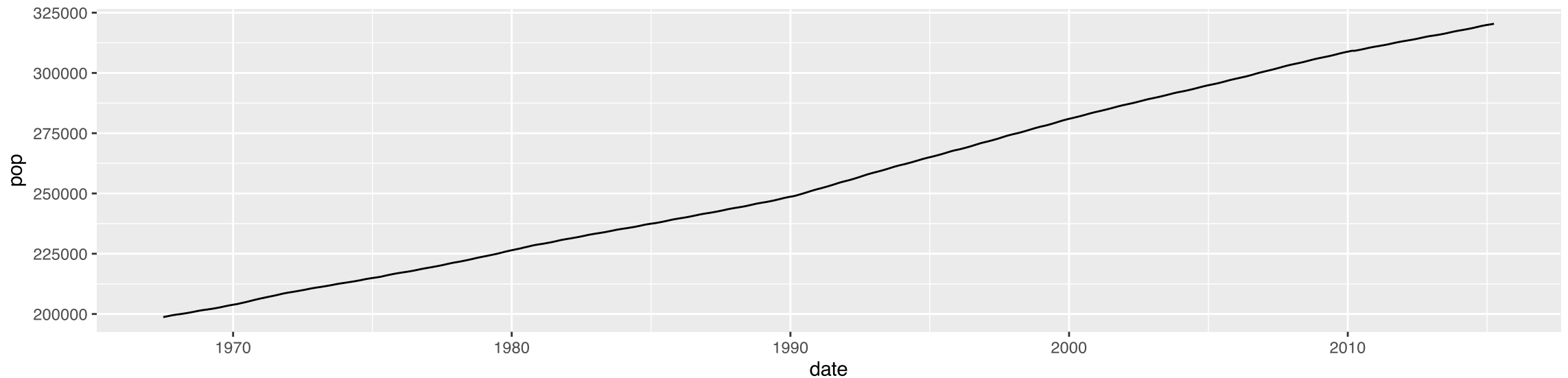
Change Over Time

Line charts are the standard way to show a series changing **over time**. You already made one in Homework 1 — let's rebuild it now that you know `ggplot` properly.

U.S. Population Over Time

1. Data: the `economics` dataset (built into `ggplot2`) · 2. Mapping: `date` on x, `pop` on y · 3. Geometry: `geom_line()`

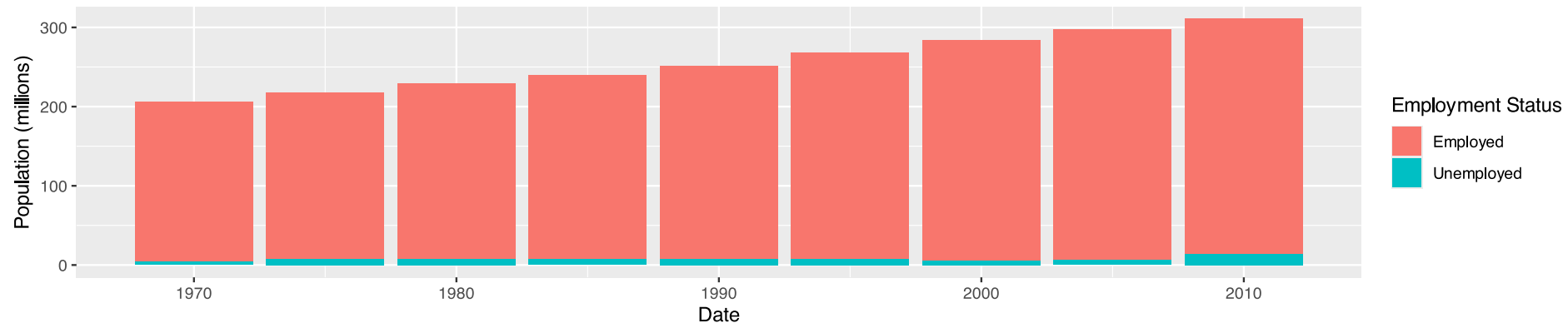
```
ggplot(economics, aes(x = date, y = pop)) +  
  geom_line()
```



Change Over Time with Stacked Bars

Stacked bar charts can show categorical **components of a whole** changing over time.

```
economics %>%  
  filter(month(date) == 12 & year(date) %% 5 == 0) %>%  
  mutate(Date = year(date), Unemployed = unemploy, Employed = pop - unemploy) %>%  
  select(Date, Unemployed, Employed) %>%  
  pivot_longer(-Date, names_to = "employment_status", values_to = "value") %>%  
  ggplot(aes(x = Date, y = value / 1e3, fill = employment_status)) +  
  geom_bar(stat = "identity") +  
  labs(y = "Population (millions)", fill = "Employment Status")
```



What are the strengths of a stacked bar chart here? The weaknesses?

Magnitude

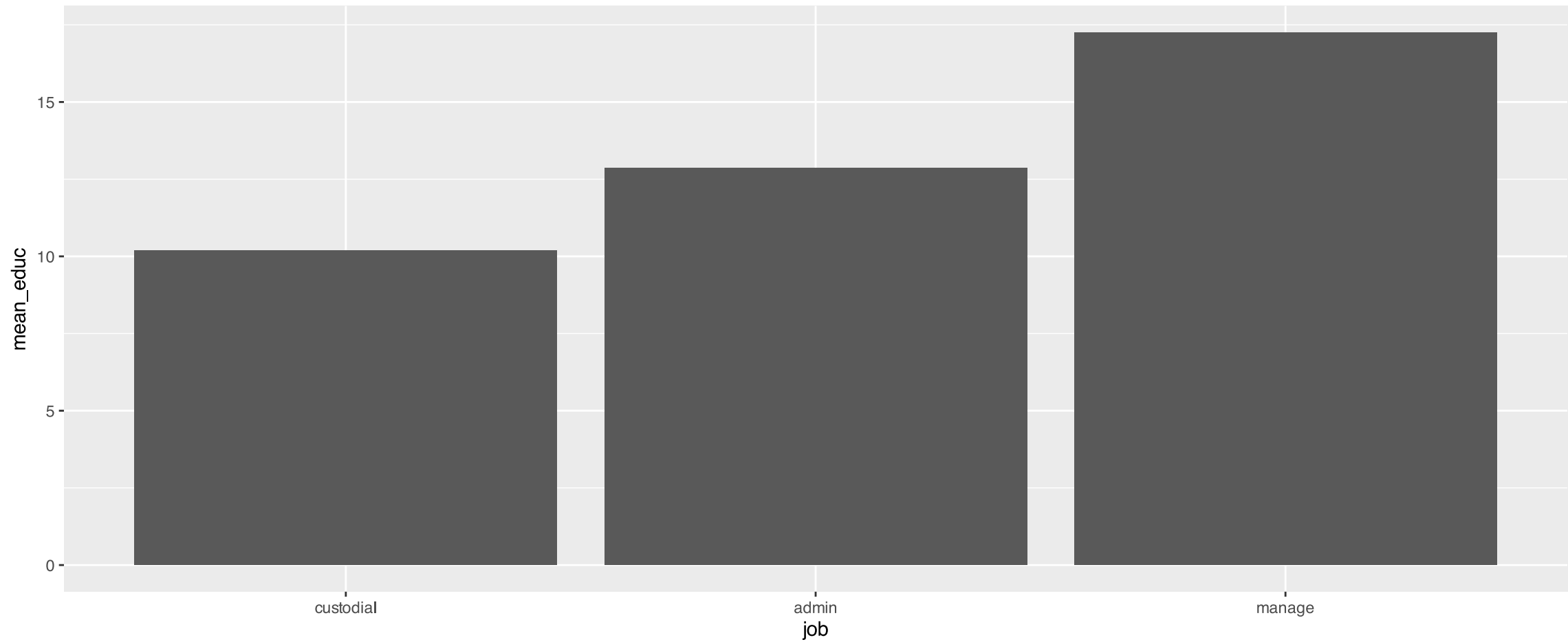
Shows size comparisons **across groups** — we just saw one example (population by employment status). **Bar charts** are the standard way to compare magnitude.

Average Education by Job

1. Data: `BankWages`, grouped by `job` ·
2. Mapping: `job` (discrete) on x, mean `education` (continuous) on y ·
3. Geometry: `geom_bar(stat = "identity")`¹

```
BankWages %>%  
  group_by(job) %>%  
  summarize(mean_educ = mean(education)) %>%  
  ggplot(aes(x = job, y = mean_educ)) +  
  geom_bar(stat = "identity")
```

¹Without `stat = "identity"`, `geom_bar()` counts rows instead of plotting a value you already computed.



Relationship Between Two Variables

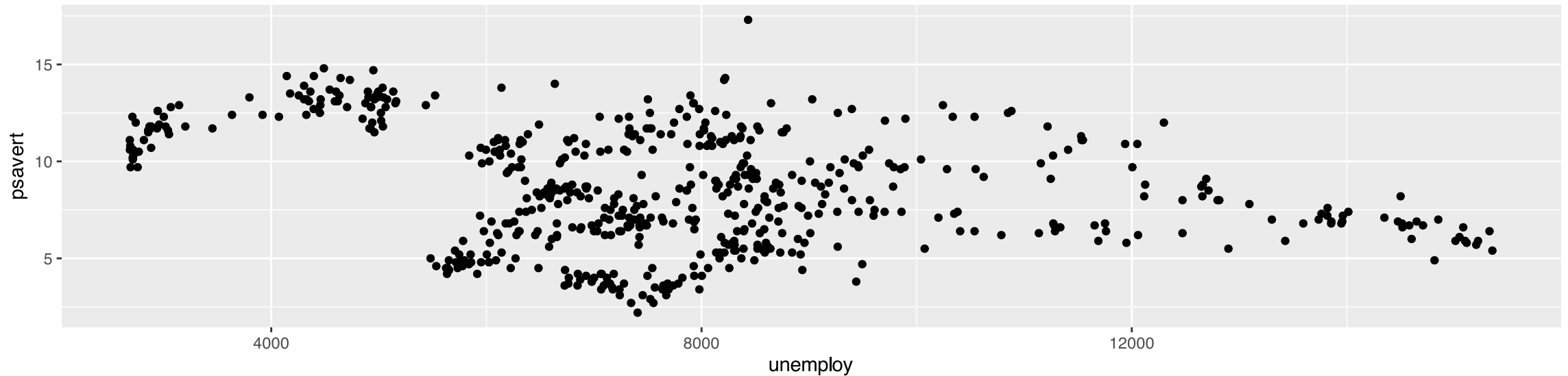
What visualization shows the relationship between two variables — say, unemployment and the savings rate?

Scatterplots: each variable gets its own axis, each observation a point.

Unemployment vs. Savings Rate

1. Data: `economics` · 2. Mapping: `unemploy` on x, `psavert` on y · 3. Geometry: `geom_point()`

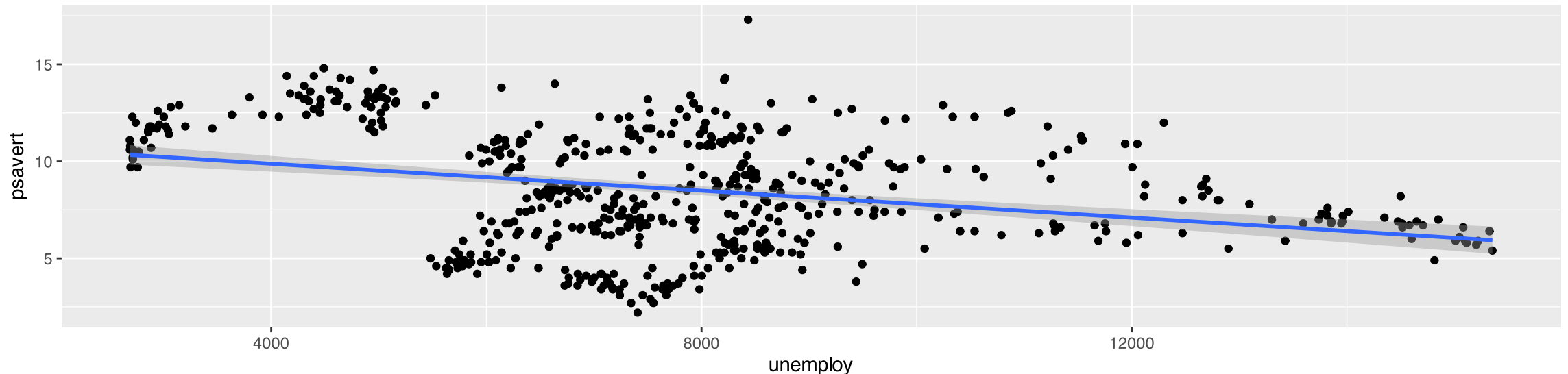
```
ggplot(economics, aes(x = unemploy, y = psavert)) +  
  geom_point()
```



Adding a Trend Line

`geom_smooth(method = "lm")` adds a line that minimizes the distance to every point in the scatterplot — the general direction of the relationship.

```
ggplot(economics, aes(x = unemploy, y = psavert)) +  
  geom_point() +  
  geom_smooth(method = "lm")
```



Breakout Session 1

Task: Use the `Guns` dataset ([AER](#)) to practice today's chart types.

1. Filter to `year == "1990"` and create a scatterplot of the violent crime rate (`violent`, x-axis) against the incarceration rate (`prisoners`, y-axis).
2. Use `case_when()` to create `inc_bracket`, a categorical variable with **low**, **middle**, and **high** income brackets, using \$12,000 and \$15,000 (`income`) as the breakpoints. Re-make the scatterplot, adding `+ geom_point(aes(color = inc_bracket))`. What changed?
3. Add a trend line with `geom_smooth()`.

Discuss: Do you see a trend? Any outliers — which state?

03

Dates in R

- 01 Recap & Agenda
- 02 Visualizing Data
- 03 Dates in R**
- 04 Manipulating Strings
- 05 Tidyverse Review
- 06 Research Process & The Project
- 07 Recap

A Quick Review of Data Types

- **Integer** (`int`) — whole numbers: -272, 0, 42 (written `42L` in R)
- **Numeric** (`num/dbl`) — decimals: 0.5, 3.17, 3.14159...
- **Character** (`chr`) — text: "keyboard", "green", "male/female"
- **Logical** (`lgl`) — TRUE/FALSE (shorthand T/F), convertible to 1/0

Dates are none of these — they're their own type, built on top of a number.

How Does a Computer Represent a Date?

Virtually every system stores a date as the number of seconds or days since some reference date (the “origin”):

- Excel: days since December 31, 1899
- Stata: days (or seconds) since January 1, 1960
- R, C, Linux, ...: seconds since January 1, 1970 (the POSIX standard)

A date is a number with special attributes and formatting layered on top.

How R Stores a Date

```
today <- Sys.Date()  
typeof(today)
```

```
[1] "double"
```

```
class(today)
```

```
[1] "Date"
```

```
attributes(today)
```

```
$class  
[1] "Date"
```

Days Since the Origin

```
as.Date(365)
```

```
[1] "1971-01-01"
```

```
as.numeric(as.Date("1971-01-01"))
```

```
[1] 365
```

```
as.numeric(today)
```

```
[1] 20721
```

Seconds Since the Origin

```
as.POSIXct(1e9, origin = "1970-01-01")
```

```
[1] "2001-09-08 21:46:40 EDT"
```

```
as.numeric(Sys.time())
```

```
[1] 1790341245
```

Why Use Date Objects?

- **Analyzing data over time** and **comparing dates** (before/after)
- **Merging datasets** on a shared date column

Today we'll **transform** raw date strings/numbers into date objects, and **format** date objects for display.

Transforming Raw Dates: Two Steps

1. **Read** in the raw data (string or numeric) — you first need to know its format
2. **Convert** it to a date object, with `as.Date()` (base R) or `lubridate`

You have to tell R which part of the string is the year, month, day, etc. — the **format string**.

The Date Conversion Key

Symbol	Meaning	Example
%d	day (01–31)	01
%a / %A	abbrev. / full weekday	Mon / Monday
%m	month (00–12)	07
%b / %B	abbrev. / full month	Jul / July
%y / %Y	2- / 4-digit year	07 / 2007

`as.Date("Jan 1 2020", format = "%b %d %Y")` reads %b as the month, %d as the day, %Y as the year.

Transforming Dates with `lubridate`

`lubridate` functions like `ymd()`, `mdy()`, `dmy()` are more compact than `as.Date()`, but only work for specific orderings — pick the function whose **letter order matches your data**.

```
mydates <- c("2020-02-07", 20200207)
ymd(mydates)
```

```
[1] "2020-02-07" "2020-02-07"
```

```
mdy("02/07/2020") # month first
```

```
[1] "2020-02-07"
```

```
dmy("07/02/2020") # day first
```

```
[1] "2020-02-07"
```

```
ymd("2020m2", truncated = 1) # missing day component is OK with `truncated`
```

```
[1] "2020-02-01"
```

Notice `/`, `-`, or no separator at all work.

When the Format Isn't Recognizable

```
as.Date("10/20/2022")
```

```
Error in `charToDate()`:  
! character string is not in a standard unambiguous format
```

Neither base R nor `lubridate` can guess a format on its own — you have to supply one, or use `mdy()`/
`dmy()` directly.

Building a Date from Components

`make_date()` combines separate year/month/day columns into one date column ([make_datetime\(\)](#) adds hours/minutes/seconds):

```
tibble(
  event = c("Kickoff", "Midterm", "Guest Speaker"),
  year  = c(2026, 2026, 2026),
  month = c(8, 10, 11),
  day   = c(28, 15, 6)
) %>%
  mutate(date = make_date(year = year, month = month, day = day))
```

```
# A tibble: 3 × 5
  event      year month   day date
  <chr>    <dbl> <dbl> <dbl> <date>
1 Kickoff    2026     8    28 2026-08-28
2 Midterm    2026    10    15 2026-10-15
3 Guest Speaker 2026    11     6 2026-11-06
```

Breakout Session 2

Task: `economics_dates.csv` stores the same eight months, encoded eight different ways.

```
economics_dates <- read_csv("data/economics_dates.csv")
economics_dates %>% select(date1, date4, date6) %>% head(3)
```

```
# A tibble: 3 × 3
  date1      date4      date6
  <date>    <chr>    <chr>
1 1967-07-01 Jul 1967 01-Jul-1967
2 1967-08-01 Aug 1967 01-Aug-1967
3 1967-09-01 Sep 1967 01-Sep-1967
```

1. Look at `date1`, `date4`, and `date6`. Identify each column's format — which part is year, month, day?
2. Convert each to a proper date object with the matching `lubridate` function.
3. Confirm the conversion worked with `glimpse()`.

Hint: `date4` is missing the day — every observation is the 1st of the month. Which argument handles a missing date component?

Formatting the Display of a Date

By default, R **displays** a date as `"yyyy-mm-dd"` — but the underlying object can be shown differently without changing its type.

When would you want a different display? Formatting an axis in a plot: `Aug 19` instead of `2019-08-01`.

format()

`format(x, format = "")` uses the same mnemonics from the Date Conversion Key to change a date's *display*:

```
today <- Sys.Date()
format(today, format = "Today is %B %d, %Y")
```

```
[1] "Today is September 25, 2026"
```

`format(date, "%A, %b %d")` → "Friday, Feb 7"

Careful: the output of `format()` is a **character string**, not a date — you lose the date type the moment you save it.

```
prettyday <- format(today, format = "Today is %B %d, %Y")
is.Date(prettyday)
```

```
[1] FALSE
```

04

Manipulating Strings

- 01 Recap & Agenda
- 02 Visualizing Data
- 03 Dates in R
- 04 Manipulating Strings**
- 05 Tidyverse Review
- 06 Research Process & The Project
- 07 Recap

Reviewing Strings

- Strings (or “characters”) are text objects — addresses, names, ID codes: "42 Wallaby Way, Sydney", "green", "male/female"
- A **string** is one piece of text; a **character vector** is a vector of strings

Why Manipulate Strings?

- **Extracting** portions of text
 - "Washington, DC", but you only want the 2-letter state code
 - Splitting a full name into first and last
- **Cleaning** datasets
 - One dataset is lower-case, the other upper-case, and you need to merge them
 - "economics" and "econ" are the same major, recorded two different ways

Base R String Functions

```
paste("Hello", "World!")
```

```
[1] "Hello World!"
```

```
paste("Hello", "World!", sep = ", ")
```

```
[1] "Hello, World!"
```

```
nchar("Hello World!")
```

```
[1] 12
```

Use `collapse` instead of `sep` to squash a whole character *vector* into one string:

```
paste(letters[1:5], collapse = "-")
```

```
[1] "a-b-c-d-e"
```

More Base R String Functions

Most follow the pattern `pattern, replacement, string`:

- `grep()` / `grepl()` — index (or `TRUE/FALSE`) of the first match
- `sub()` / `gsub()` — replace the first (or every) match
- `strsplit()` — split a string at each match
- `substr()` — the substring between two index positions
- `toupper()` / `tolower()` — change case

```
sub("l", "L", "Hello World!")
```

```
[1] "HeLlo World!"
```

```
substr("Hello World!", 7, 11)
```

```
[1] "World"
```

stringr: A Friendlier Syntax

`stringr` functions take arguments in the order `string`, `pattern`, `(replacement)` — more intuitive than base R, and vectorized over *both* the string and the vector.

<code>stringr</code>	Base R equivalent
<code>str_detect()</code>	<code>grepl()</code>
<code>str_replace()</code> / <code>str_replace_all()</code>	<code>sub()</code> / <code>gsub()</code>
<code>str_split()</code>	<code>strsplit()</code>
<code>str_sub()</code>	<code>substr()</code>
<code>str_trim()</code> / <code>str_squish()</code>	trims whitespace (leading/trailing, or all)

`stringr` ships as part of `library(tidyverse)` — no separate `library()` call needed.

`str_detect()`: Does the Pattern Occur?

```
starwars %>%  
  filter(str_detect(name, "^L")) %>%  
  select(name) %>%  
  head(3)
```

```
# A tibble: 3 × 1  
  name  
  <chr>  
1 Luke Skywalker  
2 Leia Organa  
3 Lando Calrissian
```

`str_replace()`: Fix Inconsistent Text

```
starwars %>%  
  mutate(name = str_replace(name, "-", "")) %>%  
  select(name) %>%  
  head(3)
```

```
# A tibble: 3 × 1  
  name  
  <chr>  
1 Luke Skywalker  
2 C3PO  
3 R2D2
```

Removes the hyphen from names like `C-3PO` and `R2-D2`.

`str_split()`: Break a String Apart

```
str_split(c("Cleese, John", "Jones, Terry"), ", ")
```

```
[[1]]  
[1] "Cleese" "John"  
  
[[2]]  
[1] "Jones" "Terry"
```

`str_sub()`: Extract a Substring

```
my_string <- "Howard University"  
str_sub(my_string, start = 1, end = 6)
```

```
[1] "Howard"
```

```
str_sub(my_string, start = 8)
```

```
[1] "University"
```

Regular Expressions

“Regex” is a compact language for describing text patterns — worth a first look, even if you won’t need to be fluent in it for this course.

```
test <- c("The quick brown fox jumped over the lazy dog.",  
          "I have $2400.45 in my bank account.",  
          "Today is Friday, February 17th.")
```

Character Classes

Put a class in square brackets; add a qualifier (count) in curly braces:

```
str_detect(test, "[A-Z]")      # any uppercase letter
```

```
[1] TRUE TRUE TRUE
```

```
str_which(test, "[0-9]{4}")    # a run of exactly 4 digits
```

```
[1] 2
```

```
str_subset(test, "[0-9]{2,4}") # a run of 2 to 4 digits
```

```
[1] "I have $2400.45 in my bank account." "Today is Friday, February 17th."
```

Common shorthand classes: `[:alpha:]`, `[:upper:]`, `[:lower:]`, `[:digit:]`, `[:punct:]`, `[:space:]`.

Reserved Characters

Some characters have a special meaning in regex:

- `?` zero or one · `*` zero or more · `+` one or more
- `.` any character · `^` start of string · `$` end of string

```
str_detect(test, "^I")      # starts with "I"
```

```
[1] FALSE  TRUE FALSE
```

```
str_extract(test, "[^0-9]") # first non-digit character
```

```
[1] "T" "I" "T"
```

Escaping

To search for a reserved character itself, escape it with `\\`:

```
str_extract(test, "\\$[0-9]+\\. [0-9]{2}") # a dollar amount
```

```
[1] NA          "$2400.45" NA
```

[More on regex and escaping](#)

05

Tidyverse Review

- 01 Recap & Agenda
- 02 Visualizing Data
- 03 Dates in R
- 04 Manipulating Strings
- 05 Tidyverse Review**
- 06 Research Process & The Project
- 07 Recap

Tidyverse Review: Putting the Pieces Together

Quick reminders from Lectures 2 and 4 — not new material, just muscle memory before the project work later in the lecture.

Verb	What it does
<code>select()</code> / <code>filter()</code>	choose columns / choose rows
<code>mutate()</code> + <code>case_when()</code>	make a new column, including a multi-way categorical one
<code>group_by()</code> + <code>summarize()</code>	conditional summary statistics, one row per group
<code>pivot_longer()</code> / <code>pivot_wider()</code>	reshape long ↔ wide

One Pipe, Start to Finish

```
mtcars %>%  
  filter(cyl %in% c(4, 6)) %>%  
  mutate(fuel_economy = case_when(  
    mpg > 25 ~ "High",  
    mpg > 18 ~ "Ok",  
    TRUE ~ "Low")) %>%  
  group_by(cyl, fuel_economy) %>%  
  summarize(mean_hp = mean(hp), .groups = "drop")
```

```
# A tibble: 4 × 3  
  cyl fuel_economy mean_hp  
  <dbl> <chr>         <dbl>  
1     4 High          75.5  
2     4 Ok           91.2  
3     6 Low          123  
4     6 Ok           122.
```

Every step is a verb you already know. The skill isn't memorizing more functions — it's **chaining the right ones**, in the right order, for the question in front of you.

06

Research Process & The Project

- 01 Recap & Agenda
- 02 Visualizing Data
- 03 Dates in R
- 04 Manipulating Strings
- 05 Tidyverse Review
- 06 Research Process & The Project**
- 07 Recap

Research Process: Where the Project Stands

You now have both halves of what Project Update #3 asks for:

- **Conditional summary statistics** — `group_by()` + `summarize()`, from Lecture 3
- **Data visualizations** — histograms and scatterplots, from today

Project Update #3 asks you to compute and interpret conditional statistics for your key variables, then build a histogram and one relationship chart (scatterplot, or a visual of your choosing) — and to **interpret**, not just produce, each one.

The Research Process Is Iterative

- Your first pass at a chart or a summary table is rarely your last — expect to redo it once you see what it actually shows
- **Interpretation is the point.** A chart or a table without a sentence explaining what it means doesn't answer your research question
- Lean on your mentor and project buddy — email at least once a week, and bring specific questions, not just “is this right?”

This Week's Timeline

- **HW #5 — due October 1.** Today's material: plots, dates, strings.
- **Project Update #3 — due October 3.** Conditional summary statistics and two visualizations, on your own project data.

Stuck on *your* data (not the homework)? That's exactly what your mentor and project buddy are for — ask before the deadline, not after.

07

Recap

- 01 Recap & Agenda
- 02 Visualizing Data
- 03 Dates in R
- 04 Manipulating Strings
- 05 Tidyverse Review
- 06 Research Process & The Project
- 07 Recap**

Recap

- **Visualization** has a grammar — data, mapping, geometry — and the relationship you're showing (distribution, change over time, magnitude, or between two variables) points to the right geometry
- **Dates** are numbers underneath — `lubridate` converts raw strings/numbers into date objects, and `format()` changes only how they're *displayed*
- **Strings** can be extracted, replaced, and split with `stringr`, and regex gives you a compact way to describe a pattern
- Reviewed the **tidyverse** toolkit from Lectures 2 and 4
- Connected today's tools directly to **Project Update #3**

Links & Further Reading

- [R for Data Science: Layers \(the `ggplot2` grammar\)](#)
- [ggplot2 cheat sheet](#)
- [R for Data Science: Dates and Times](#)
- [lubridate cheat sheet](#)
- [R for Data Science: Regular Expressions](#)
- [stringr cheat sheet](#)

Coming up: hypothesis testing and correlation.